Zappit · 내부 검토 문서 · 2026년 11월 · 2판

로그인 방식 변경 설계안

"보이지 않는 계정" + QR 연결.
로그인 화면을 없애고 QR 한 번으로 연결하되, 지금의 계정 구조(uid)는 그대로 쓴다.

확정된 방향

- **로그인 화면 없음.** 앱과 확장이 사용자 몰래 Firebase **익명 계정**을 만들고, QR로 둘을 같은 계정에 붙인다. 출시 전이므로 Apple/Google·Safari 웹 로그인 코드는 모두 걷어낸다.
- **1차는 QR만.** 6자리 숫자 입력은 넣지 않는다. 폰은 계정당 3대까지(출시부터), 브라우저는 제한 없음.
- **서버는 Firebase만으로 충분.** Render 같은 별도 서버는 필요 없다. (2장)
- **Apple/Google 백업 기능은 만들지 않는다.** 폰과 확장을 둘 다 잃으면 새 계정으로 시작하고, 구독은 "Restore Purchases"로 되찾는다. (3장)
- **확장의 60일 로그아웃은 없앤다.** 대신 60일 동안 앱 연결이 없으면 RTDB 연결만 끊고, 아이콘을 누르면 다시 붙는다. RTDB 동시 접속 15만에서 경고. (과제 ⑥)
- **QR을 찍은 휴대폰이 그 PC를 가져간다.** 다른 휴대폰이 쓰던 PC라도 확인 창 없이 바로 연결된다. 원래 휴대폰은 다시 찍으면 된다. (3-1, 과제 ①)
- **맛보기 방어의 중심은 OS 서버에 남는 기기 표시**(iOS DeviceCheck, Android device recall). 앱과 확장을 모두 지우거나 기기를 초기화해도 다시 받을 수 없다. (5장)

이 문서의 순서

1. **지금 구조와 새 방식 한눈에** — 무엇이 그대로이고 무엇이 바뀌나
2. **연결 흐름과 서버** — QR 링크, OneLink, Firebase만으로 되는 이유, 보안
3. **연결 규칙과 재설치** — 누가 누구 계정에 합류하나, 둘 다 잃었을 때
4. **구독** — 복원 순서, 구독 공유 문제와 폰 수 제한
5. **맛보기(Pro Preview) 방어** — "기기 식별 값 확보 방안" 평가와 추천 구조 (가장 길게)
6. **화면 예시와 보안 장치**
7. **남는 과제** — 다른 휴대폰이 쓰던 PC, 재설치, 60일 미사용 처리 등
8. **결론과 진행 순서**

용어 몇 개

- **uid**: 계정 번호. Firebase가 계정마다 붙이는 고유한 문자열. 지금 Zappit의 거의 모든 데이터가 이 번호 아래에 저장됩니다.
- **익명 계정**: 이메일·비밀번호 없이 Firebase가 만들어 주는 계정. 사용자는 아무것도 입력하지 않지만 uid는 생깁니다.
- **custom token(로그인 표)**: 서버가 "이 기기는 uid A로 로그인해도 된다"고 발급하는 1회용 표. 지금 Safari 확장 로그인(`extension_token`)에 이미 쓰고 있습니다.
- **RevenueCat**: 구독 관리 서비스. 지금처럼 "uid = 구독자 번호"로 씁니다.

1. 지금 구조와 새 방식 한눈에

지금은 폰과 확장이 **같은 Apple/Google 계정으로 각각 로그인**해서 같은 uid를 갖습니다. 그 uid가 모든 것의 열쇠입니다.

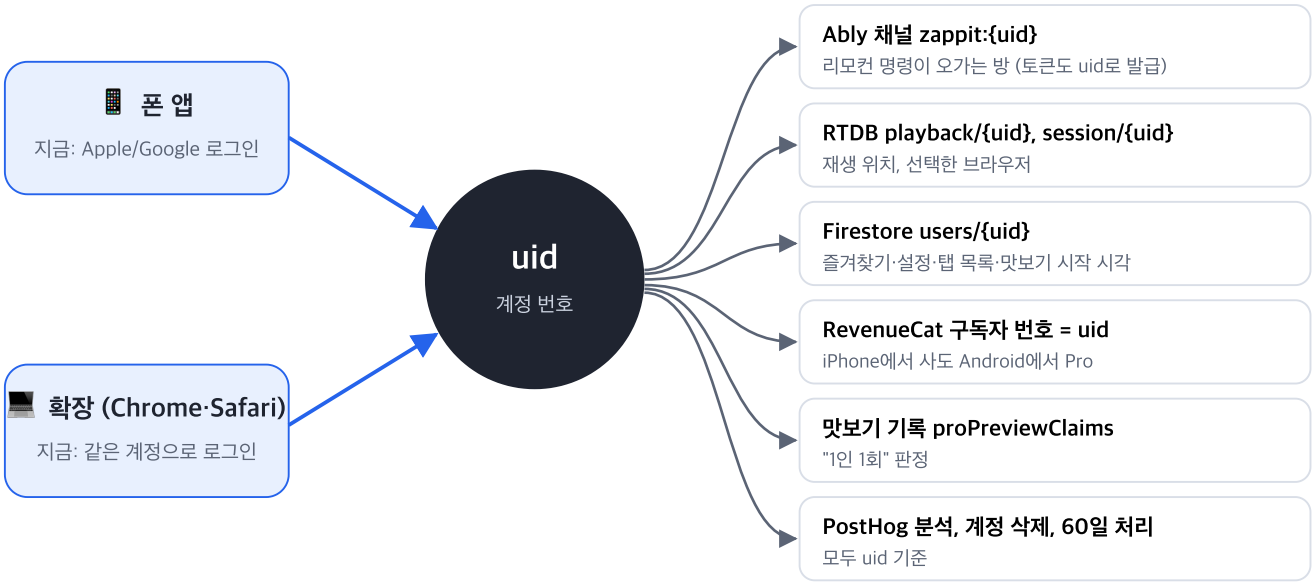


그림 1. uid 하나에 여섯 가지가 매달려 있다. 새 방식은 오른쪽 여섯 가지를 그대로 두고, 왼쪽 "uid를 얻는 방법"만 바꾼다.

그대로

- Ably 채널 `zappit:{uid}` 와 `ably_token`
- RTDB·Firestore 저장 위치와 보안 규칙
- RevenueCat 구독자 번호 = uid
- 여러 브라우저, 브라우저 선택 화면
- 맛보기 시작 시점 (처음 연결될 때)

새로 만들거나 바뀌는 것

- 서버: 연결 함수 `pair_start` , `pair_claim`
- 앱: 익명 로그인, QR 스캔 화면 (로그인 화면 대체), 연결 해제
- 확장: 익명 로그인, QR 팝업, "연결된 폰" 목록
- 맛보기: DeviceCheck·device recall·폰 표식, App Check
- zappit.tv: `/p/<코드>` 링크와 앱 링크 설정 파일
- 확장 60일: 로그아웃 대신 연결만 끊음

서버 함수	새 방식에서
`ably_token`	그대로. 익명 계정도 uid가 있으므로 똑같이 자기 방 토큰을 받는다
`pro_preview_start`	기기 판정(DeviceCheck·device recall)과 폰 표식 확인을 추가 (5장)
`extension_token`	삭제. 출시 전이라 기존 사용자가 없고, Safari도 QR로 연결한다 (`zappit.tv/auth` 페이지도 불필요)
`delete_account`	앱 메뉴 없음. 사용자가 계정을 만드는 화면이 없으므로 따로 안내하지 않는다. 함수는 문제가 생기면 대응할 수 있게 남김

2. 연결 흐름과 서버

사용자가 보는 것

1. 앱을 연다 → 바로 "PC의 QR을 찍으세요" 화면
2. PC 확장 팝업에 QR이 떠 있다
3. 찍는다 → "연결됨!"

로그인, 이메일, 비밀번호, 동의 화면 없음.

뒤에서 일어나는 것

1. 앱과 확장이 각자 처음 실행될 때 **익명 계정**을 만든다
2. 확장이 서버에서 **5분짜리 1회용 연결 코드**를 받아 QR로 보여준다
3. 앱이 코드를 서버에 내면, 서버가 확장에게 **앱과 같은 uid로 로그인할 표**를 준다

2-1. 처음 연결 흐름

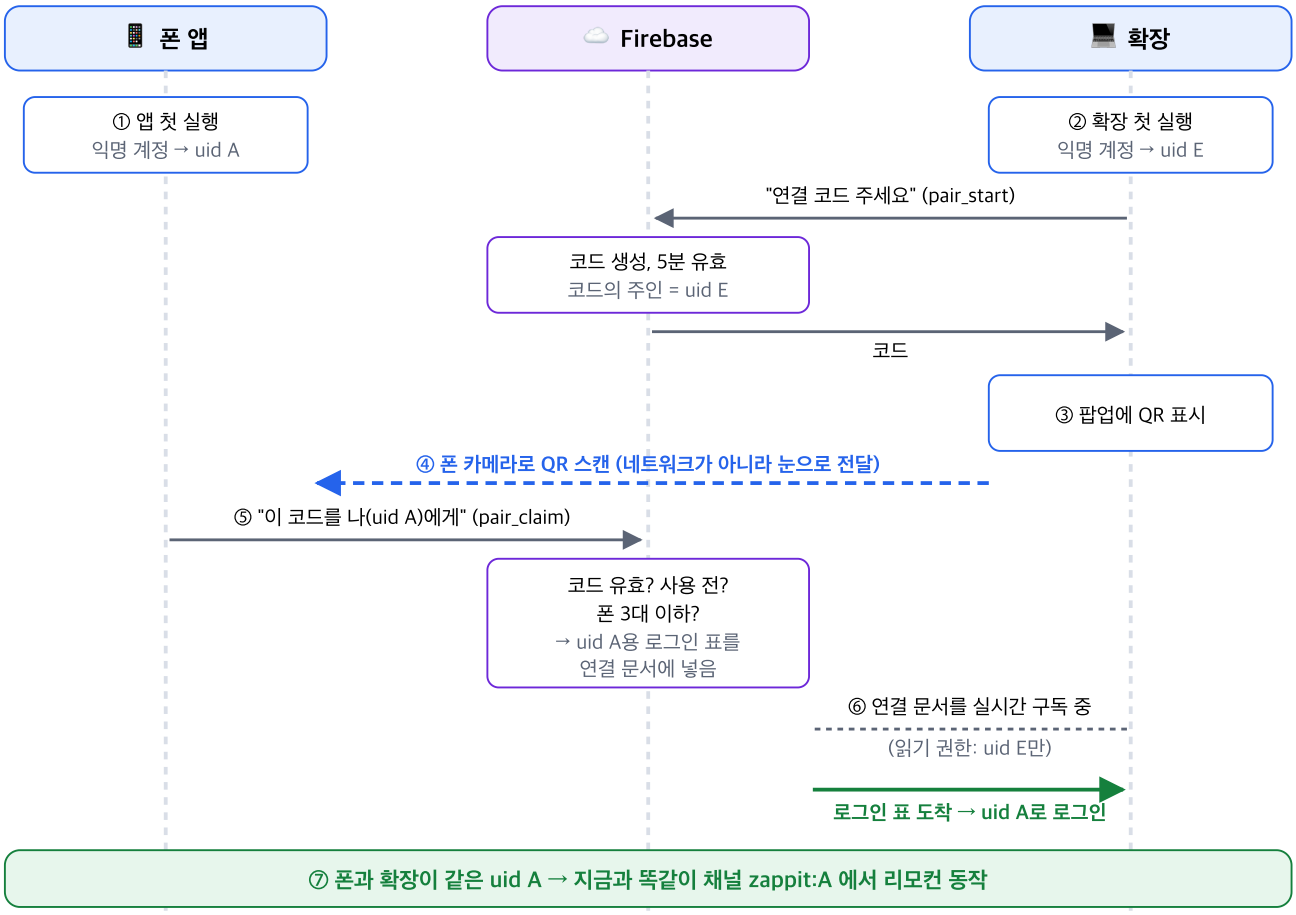


그림 2. 연결 흐름. 확장은 2초마다 묻는 대신 Firestore 문서를 실시간으로 지켜보고, 그 문서는 코드를 만든 확장(uid E)만 읽을 수 있다. 다 쓴 uid E는 나중에 정리한다.

2. 연결 흐름과 서버 (계속)

2-2. QR 안의 링크와 OneLink

앱이 없는 사람이 기본 카메라로 찍으면 스토어로 보내야 합니다. 이미 만든 OneLink(`onelink.to/ngsjzu`)는 **"스토어로 보내는 마지막 단계"로만** 씁니다. QR 자체에 OneLink를 넣으면 안 되는 이유는 아래와 같습니다.

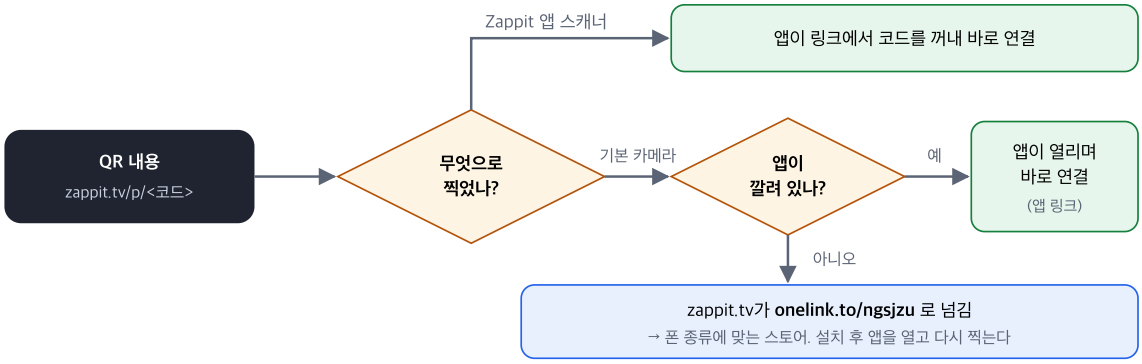


그림 3. QR은 우리 도메인(zappit.tv) 링크로 만들고, 앱이 없을 때만 OneLink로 넘긴다.

QR에 OneLink를 바로 넣으면	문제
앱이 있는 사람이 기본 카메라로 찍음	앱이 아니라 스토어 페이지가 열린다. 스토어의 "열기"를 눌러도 코드는 전달되지 않는다
앱으로 바로 여는 기능	iOS Universal Link·Android App Link는 **우리가 관리하는 도메인**에 설정 파일을 둬야 한다. onelink.to는 우리 도메인이 아니다

할 일 (zappit.tv, Render)

- `/.well-known/apple-app-site-association` , `/.well-known/assetlinks.json` 파일을 올린다 (앱 링크 설정).
- Render 리다이렉트 규칙: `/p/*` → `https://onelink.to/ngsjzu` . 웹 페이지를 따로 만들 필요는 없다.
- 설치하는 동안 코드가 만료될 수 있지만, 팝업이 열려 있는 동안 확장이 새 코드로 계속 바꾸므로 다시 찍으면 된다.

2-3. 서버는 Firebase만으로 충분하다

필요한 것	Firebase에서
코드 발급, 코드 붙이기, 로그인 표 발급	Cloud Functions callable (지금의 `ably_token` · `extension_token` 과 같은 구조)
코드 저장과 만료	Firestore `pairings/{code}` . 함수가 `expiresAt` 을 확인하고 TTL 정책이 지운다
확장이 결과를 기다림	Firestore 실시간 구독. 보안 규칙으로 코드 주인(uid E)만 읽기
확장용 로그인 표	custom token. `extension_token` 에서 이미 쓰고 있어 서명 권한 설정이 끝나 있다
가짜 앱의 호출 차단	App Check (App Attest / Play Integrity)
기기 판정 (5장)	함수에서 Apple DeviceCheck·Google Play Integrity 서버를 부른다. 서버를 따로 둘 필요 없음

Render는 zappit.tv 정적 사이트(링크·설정 파일)에만 쓴다.

2. 연결 흐름과 서버 (계속)

2-4. 왜 "코드를 만든 확장만" 결과를 받아야 하나

QR 속 코드는 화면에 보이므로 **원래 공개된 값**입니다. 만약 코드만 알면 "연결 결과(로그인 표)"를 받을 수 있다면 이런 일이 생깁니다.

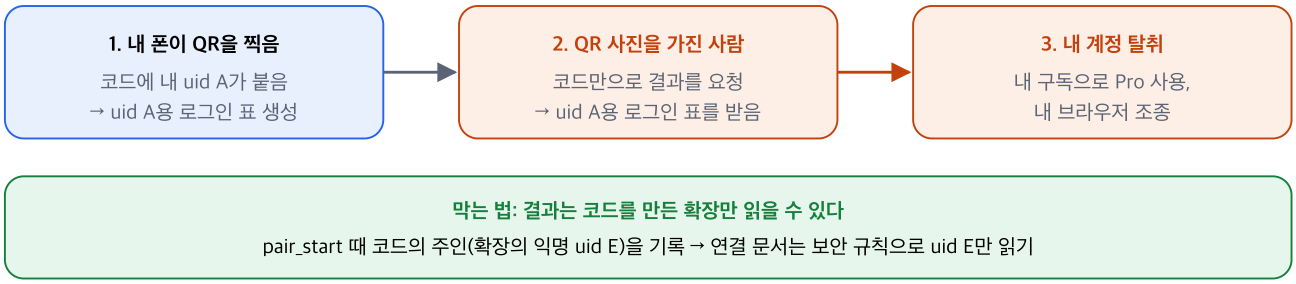


그림 4. 1판의 "비밀값은 확장만 보관"은 이 뜻이었다. 확장이 먼저 익명 로그인하면 그 uid가 비밀값 역할을 하므로 비밀값이 따로 필요 없다.

반대 방향: QR 선점

남이 내 PC의 QR을 **나보다 먼저** 찍으면 내 PC가 그 사람 계정에 붙습니다(그 사람이 내 PC를 조종). 대책:

- 코드는 1회용이라, 내가 뒤이어 찍으면 "이미 사용된 코드"가 떠서 바로 알 수 있다.
- 연결되면 확장 팝업에 "iPhone (방금 연결)"처럼 연결한 폰 이름이 보이고, "연결된 폰" 목록에서 해제할 수 있다.
- 남이 내 화면을 5분 안에 직접 볼 수 있어야 하므로 현실적인 위험은 작다.

2-5. 합류할 때도 같은 원리

처음 연결이 아니라 **폰이 확장의 계정으로 들어가는 경우**(3장: 두 번째 폰, 앱 재설치)에는 서버가 **폰에게** 확장 계정(uid)의 로그인 표를 줍니다. 방향만 반대이고, 같은 함수·같은 문서로 처리합니다.

3. 연결 규칙과 재설치

3-1. 연결 규칙: "새로 온 쪽이 기존 쪽에 합류"

QR을 찍을 때 폰과 확장 중 **누가 누구의 계정으로 들어갈지** 정하는 규칙입니다.

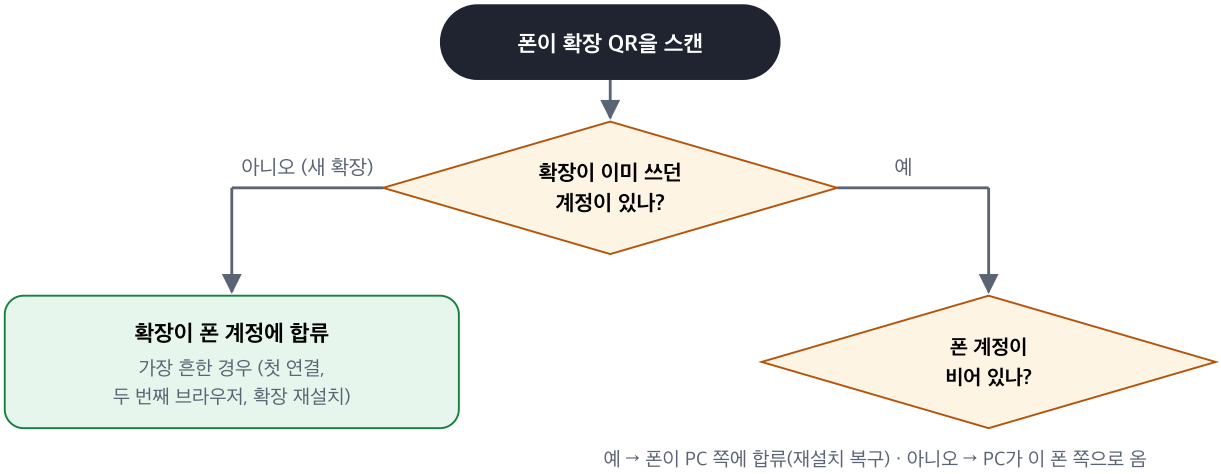
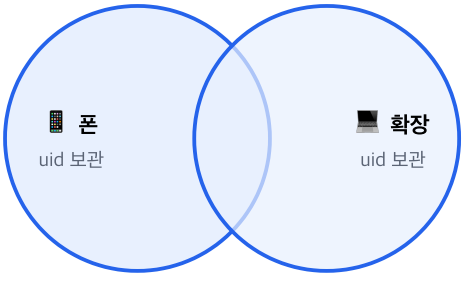


그림 5. 연결 규칙. 확인 창은 없다. "비어 있다" = 맛보기·바꾼 설정이 하나도 없는 새 익명 계정. 구독이 "복원"으로만 들어온 계정도 비어 있는 것으로 본다 (4-1).

상황	규칙 적용 결과
처음 쓰는 사람	새 확장 → 폰 계정에 합류
Chrome에 이어 Safari 추가	Safari 확장(새것) → 폰 계정에 합류. 두 브라우저가 같은 방
두 번째 폰(Android) 추가	새 폰(비어 있음) → 확장 계정에 합류. 두 폰이 같은 계정 = 구독 공유
앱 재설치	재설치한 폰(비어 있음) → 확장 계정에 합류 = **예전 계정 그대로 복귀**
확장 재설치	새 확장 → 폰 계정에 합류
쓰던 휴대폰으로 다른 휴대폰이 쓰던 PC를 찍음	PC가 이 휴대폰 쪽으로 옴. 원래 휴대폰 목록에서는 빠지고, 그 휴대폰으로 다시 찍으면 돌아감 (과제 ①)
폰을 새로 삼 (iPhone → Android 포함)	새 폰 → 확장 계정에 합류. 구독은 uid에 붙어 있어 **스토어가 달라도 Pro 유지**

3-2. 폰과 확장이 서로의 "열쇠 보관함"



둘 다 잃었을 때 (백업 없음)

- 새 계정으로 시작
- 구독: "Restore Purchases"로 되찾음
  (같은 스토어일 때. 4-1 참고)
- 설정·즐겨찾기: 잃음 (감수)
- 맛보기: 기기 판정으로 재지급 안 함

그림 6. 백업 기능은 만들지 않는다. 둘 다 지운 사용자는 대부분 그만 쓰려는 사용자이고, 잃는 것은 설정 정도다.

4. 구독

4-1. 재설치 후 "Restore Purchases"와 QR, 순서가 중요한가

RevenueCat은 구독자 번호가 uid이므로, 새 익명 계정에는 처음엔 구독이 없습니다. "Restore Purchases"를 누르면 스토어 영수증이 지금 계정으로 **이사**합니다.

상황	순서	결과
폰·확장 **둘 다** 재설치	복원 → QR	폰 계정에 구독이 들어오고, 새 확장이 폰 계정에 합류. 문제없음
	QR → 복원	확장이 폰 계정에 합류한 뒤 복원으로 구독이 들어옴. 문제없음
앱만 재설치 (확장은 예전 계정 A)	QR만	폰이 A로 합류 → 구독·설정·맛보기 모두 그대로. 복원할 필요도 없음
	복원 → QR	영수증이 새 계정 B로 이사해 A에는 구독이 없어짐. → 규칙(확정): **"복원으로만 생긴 구독은 비어 있는 것으로 본다"** → 폰이 A로 합류하고, 합류 직후 앱이 `syncPurchases` 로 구독을 한 번 다시 동기화해 영수증을 A로 되돌림

확인할 것

- RevenueCat 대시보드의 복원 동작이 **"Transfer to new App User ID"**(영수증 이사)여야 위 표대로 동작합니다. (설정 확인 완료)
- **자동 동기화는 `syncPurchases` (확정).** `restorePurchases` 는 iOS에서 Apple ID 로그인 창을 띄울 수 있어, 사용자가 직접 누르는 "Restore Purchases" 버튼에만 쓴다. `syncPurchases` 도 Transfer 설정대로 영수증을 옮기는지 샌드박스에서 확인한다.
- 둘 다 잃고 **스토어까지 바뀐 경우**(iPhone에서 구독 → Android로 바꾸면서 확장도 삭제)에는 복원할 수 없습니다. 백업이 없으므로 감수하는 경우입니다. 확장이 살아 있으면 스토어가 달라도 유지됩니다(3-1).

4. 구독 (계속)

4-2. 구독 하나를 여럿이 나눠 쓸 수 있나

번갈아 쓰는 것은 가능하고, 동시에 쓰는 것은 사실상 불가능합니다. 같은 계정이면 모두 Pro이지만, 리모컨이 조종하는 브라우저가 계정에 하나뿐이기 때문입니다.

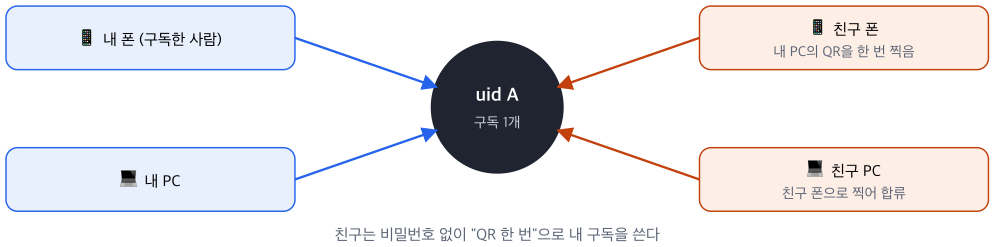


그림 7. 지금은 Apple/Google 비밀번호를 알려 줘야 나눌 수 있지만, QR은 문턱이 훨씬 낮다. 단, 같은 계정은 한 번에 한 사람만 쓸 수 있다.

동시에는 못 쓴다

조종할 브라우저 (`session/{uid}/selectedInstanceId`)와 재생 상태(`playback/{uid}`)가 **계정당 하나**다. A가 자기 PC의 YouTube를, B가 자기 PC의 TikTok을 보다가 한 사람이 브라우저를 고르면 다른 사람의 리모컨도 그 브라우저로 바뀐다. 탭·브라우저 목록도 섞인다. 나눠 쓰려는 사람에게 큰 불편이라 공유를 저절로 막아 준다.

보조 대책: 폰 수 제한 (확정)

`pair_claim` 이 계정당 **폰 3대**까지만 붙인다(확정). 브라우저 수는 제한하지 않는다. 폰은 5장의 폰 표식으로 세므로 같은 폰을 재설치해도 한 대다. 넘으면: "이 PC에는 이미 휴대폰 3대가 연결되어 있어요. 이 휴대폰을 연결하려면 PC의 Zappit 확장에서 쓰지 않는 휴대폰의 연결을 해제해 주세요."

남는 것은 "번갈아 쓰기"뿐이고, 폰 수 제한은 그 범위를 줄인다.

5. 맛보기(Pro Preview) 방어

왜 중요한가

일주일마다 맛보기를 다시 받을 수 있으면 아무도 결제하지 않습니다. 로그인이 없어지면 지금의 "로그인 계정 ID·이메일 해시"를 쓸 수 없으므로, 새 방어선이 반드시 필요합니다.

5-1. 다시 받으려는 사람은 무엇을 지우나

맛보기를 다시 받으려는 사람은 **앱과 확장을 모두 지우고 다시 설치**합니다. 그러면 남는 단서는 아래와 같습니다.

단서	앱+확장 재설치 후	기기 초기화 후	비고
uid (익명 계정)	바뀜	바뀜	새 계정이 만들어짐
확장 설치 ID	바뀜	바뀜	재설치하면 새 값. 단서로 못 씀
백업 계정 해시	—	—	로그인이 없으므로 없음
iOS Keychain 표식	남음*	사라짐	*현재 동작일 뿐 Apple이 보장하는 동작은 아님
Android ANDROID_ID	같음	바뀜	Android 8.0 이상, 앱 서명 키·사용자별 값
iOS DeviceCheck 비트	남음	남음	Apple 서버에 저장. 초기화("모든 콘텐츠 및 설정 지우기")에도 유지
Android device recall 비트	남음	남음	Google 서버에 저장. 베타, 에뮬레이터 미지원

정리하면 믿을 수 있는 것은 **폰에 묶인 단서뿐**이고, 그중에서도 **OS 회사 서버에 남는 비트**가 가장 강합니다.

5-2. "기기 식별 값 확보 방안" 평가

방향(폰 표식으로 막는다)은 맞지만, 과장된 부분이 있고 가장 강한 도구 두 가지가 빠져 있습니다.

문서의 주장	판정	설명
iOS Keychain은 앱을 지워도 남는다	대체로 맞음	지금은 남지만 Apple 문서가 약속한 동작이 아니다. iOS 10.3 베타에서 지우도록 바뀌었다가 되돌린 적이 있다
Keychain은 공장 초기화에서만 사라진다	맞음	기기 백업·iCloud 키체인으로 새 폰에 따라가는 경우도 있다(방어에는 유리)
ANDROID_ID는 재설치해도 같다 (8.0+)	맞음	공장 초기화하면 바뀐다
Auto Backup이 기록을 되살린다	믿기 어려움	사용자의 백업 설정, 백업 시점(Wi-Fi·충전·대기 중)에 따라 다르다. 보조 수단일 뿐
"95% 방어", "완벽 차단", "업계 표준"	과장	근거 없는 수치다
(빠진 점) 표식의 진위	누락	Keychain 값·ANDROID_ID는 **앱이 스스로 말하는 값**이다. 함수를 직접 부르며 아무 값이나 보내면 뚫린다 → **App Check 필수**
(빠진 점) DeviceCheck · device recall	누락	Apple·Google 서버가 기록을 들고 있어 재설치·초기화로 지울 수 없다. 이 문제를 위해 만든 기능이다

5. 맛보기 방어 (계속)

5-3. 추천 구조: 세 겹

1겹 (주 방어) · OS 서버의 기기 비트

iOS DeviceCheck (기기당 2비트) · Android Play Integrity device recall (기기당 3비트)
"이 기기는 맛보기를 썼다"를 Apple·Google 서버에 기록. 재설치·초기화로 지울 수 없고, 우리 서버만 바꿀 수 있다

2겹 (보조) · 폰 표식

iOS Keychain에 넣은 임의 값 · Android ANDROID_ID → 해시만 proPreviewClaims에 저장
device recall이 아직 베타이거나 판정이 안 나오는 기기를 위한 대비. 폰 수 제한(4-2)에도 쓴다

3겹 (진위 확인) · App Check

iOS App Attest · Android Play Integrity. pro_preview_start·pair_claim은 진짜 Zappit 앱의 호출만 받는다
가짜 표식을 보내는 스크립트, 변조된 앱, 에뮬레이터를 막는다

그림 8. 1겹이 핵심이고, 2·3겹은 1겹의 빈틈(베타, 가짜 호출)을 메운다. 모두 Cloud Functions 안에서 처리한다.

5-4. 맛보기 시작 판정 흐름 (`pro_preview_start`)

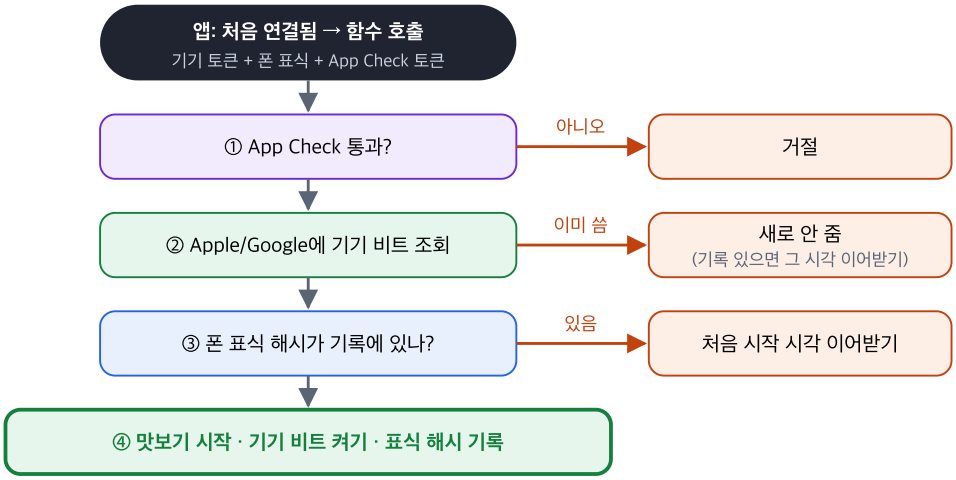


그림 9. 기기 비트가 판정되지 않는 경우(device recall 미지원 기기 등)에는 ③으로 판단한다.

5. 맛보기 방어 (계속)

5-5. 수법별로 막히는가

다시 받으려는 수법	폰 표식만 (문서 방안)	추천 구조 (세 겹)	빈도
앱만 재설치	막음	막음	흔함
앱 + 확장 모두 재설치	막음	막음	흔함
Android 설정에서 앱 데이터 삭제	막음	막음	가끔
iPhone 초기화	뚫림	막음	드묾
Android 공장 초기화	뚫림	막음*	드묾
함수를 직접 불러 가짜 표식 전송	뚫림	막음	드묾
에뮬레이터·루팅 기기	뚫림	대부분 막음	드묾
다른 폰으로 받기	폰당 1회	폰당 1회	가끔

* device recall은 Play에서 받은 계정(Play licensed)이어야 판정이 나온다. 판정이 안 나오면 ANDROID_ID(2겹)로 판단하므로, 그 경우 공장 초기화는 뚫린다. "다른 폰"은 어떤 방법으로도 막을 수 없고 막을 필요도 없다(폰 한 대마다 7일).

5-6. 구현할 때 알아 둘 것

항목	내용
DeviceCheck	앱이 Apple에서 기기 토큰을 받아 함수로 보낸다. 함수는 Apple 서버에서 비트를 읽고 쓴다(개발자 계정의 DeviceCheck 키 .p8 필요). 시뮬레이터에서는 안 된다. **직접 구현(확정)**: `device_integrity` 같은 제3자 패키지 대신 MethodChannel로 iOS(DeviceCheck 토큰)·Android(Play Integrity 토큰) 네이티브 코드를 짠다.
비트는 팀 단위	DeviceCheck 비트는 **앱이 아니라 개발자 팀 단위**로 공유된다. 같은 팀의 다른 앱이 쓰고 있다면 비트를 나눠 정해야 한다.
device recall	**1차부터 사용(확정).** Play Integrity API를 켜고(Play Console·Google Cloud 연결), 함수가 판정 토큰을 해석한 뒤 비트를 쓴다. 쓴 값은 다음 요청부터 보인다. 아직 베타라 정식 출시 전 상태를 다시 확인한다.
App Check	앱이 부르는 `pro_preview_start` · `pair_claim` 에 적용한다. 확장(`pair_start`)은 웹용 App Check가 확장 환경에 맞지 않을 수 있어 호출 횟수 제한으로 보호한다.
중고폰	전 주인이 그 폰으로 **Zappit 맛보기를 받았던 경우에만** 새 주인이 못 받는다. 비트는 우리 팀이 켠 것만 남기 때문이다. 아주 드물어 별도 규칙은 두지 않는다.
스토어 신고	App Store 개인정보 라벨("Device ID"), Play 데이터 보안 섹션("기기 또는 기타 ID")은 스토어 필수 항목이라 신고한다. 원래 값이 아니라 해시만 저장한다. 데이터 삭제 요청 안내는 어디에도 따로 두지 않는다(확정).

6. 화면 예시와 보안 장치



확장 팝업 (연결 전)　　앱 첫 화면 (로그인 화면 대신)

그림 10. 확장 팝업과 앱 첫 화면 스케치. 실제 디자인은 디자인 시안에 맞춰 따로 만든다. (화면 문구는 실제로는 영어)

보안 장치

장치	막는 것
연결 코드 5분 만료 · 1회용	QR 사진 유출. 찍힌 사진은 5분 뒤, 혹은 한 번 쓰면 끝
연결 결과는 코드를 만든 확장만 읽음	코드를 아는 남이 로그인 표를 가로채는 것 (2-4)
"연결된 폰" 목록 · 해제, "연결 초기화"	예전에 연결한 남의 폰 정리 (공용 PC, 중고 폰, QR 선점)
계정당 폰 3대 (브라우저는 제한 없음)	구독 하나를 여럿이 나눠 쓰는 것 (4-2)
App Check (앱이 부르는 함수)	가짜 앱·스크립트의 호출, 맛보기 가짜 표식

"해제"는 계정 단위 토큰 취소로는 할 수 없습니다(같은 uid라 확장까지 끊김). 그래서 폰 앱이 "연결된 폰" 목록을 **켜져 있는 동안 계속 듣고**, 자기가 빠지면 그 자리에서 스스로 연결을 끊습니다(앱 시작 때와 네트워크가 다시 연결될 때도 확인). 같은 uid라 서버에서 특정 폰만 막을 수는 없으므로, 고친 앱까지 막는 장치는 아닙니다. 정상 앱이면 이것으로 충분합니다. 공용 PC를 통째로 정리할 때는 "연결 초기화"(확장을 새 계정으로)를 씁니다.

7. 남는 과제

과제 지도 (가로: 얼마나 자주 생기나 · 세로: 생기면 얼마나 아픈가)



그림 11. 오른쪽 위일수록 먼저 설계해야 한다.

과제	한 줄 요약	상태
①	다른 휴대폰이 쓰던 PC를 찍으면, 찍은 휴대폰이 그 PC를 가져간다	확정
②	재설치로 폰의 익명 계정이 사라져도 QR 한 번으로 복귀	자동 복구
③	폰과 확장을 모두 잃으면 새로 시작 (구독은 복원)	감수
④	공용 PC·나눠 쓰기: 폰 수 제한, "연결된 폰" 목록	제한으로 완화
⑤	출시 전이라 전환 없음. 로그인 코드 걷어내기, More의 Log out → 연결 해제	확정
⑥	확장 60일: 로그아웃 대신 RTDB 연결만 끊고 아이콘으로 복귀. 동시 접속 15만 경고	확정
⑦	버려진 익명 계정 정리	1차에는 안 함

7. 남는 과제 (계속)

과제 ① 다른 휴대폰이 쓰던 PC를 찍을 때 확정 · 찍은 휴대폰이 가져감

언제 생기나: 이미 쓰던 휴대폰(iPhone, 집 PC와 연결)으로, 다른 휴대폰(Galaxy S24)이 쓰던 회사 PC의 QR을 찍을 때. 사용자가 원하는 것은 "지금 이 휴대폰으로 이 PC를 조종하는 것"뿐이므로, **확인 창 없이 PC를 이 휴대폰 쪽으로 가져옵니다.** 첫 연결과 똑같은 흐름입니다.



그림 12. 사용자에게 "묶음"을 설명하지 않는다. 연결이 사라진 쪽은 QR을 다시 찍으면 된다.

항목	PC가 옮겨 갈 때
구독	구독은 휴대폰 쪽에 있으므로 어느 쪽도 잃지 않는다. 회사 PC는 이제 iPhone의 Pro를 따른다
맛보기	PC를 옮겨도 새 맛보기가 생기지 않는다 (기기 비트·폰 표식 기준)
즐겨찾기·설정	휴대폰 쪽에 있으므로 어느 쪽도 잃지 않는다. 회사 PC는 iPhone의 설정을 쓴다
남의 PC 가로채기	QR이 1회용·5분이라 그 화면을 직접 봐야 한다. 지금과 같은 수준 (2-4)

새로 필요한 처리

- **떠나는 쪽 정리:** PC가 옮겨 갈 때 Galaxy 쪽 데이터에서 이 브라우저를 지운다(탭 목록 문서, 선택된 브라우저였다면 선택 해제). 안 지우면 Galaxy 목록에 꺼진 브라우저로 남는다.
- **Galaxy에 한 번 알림(확정):** Galaxy 앱을 다음에 열 때 상단 알림 한 번. "Chrome · Mac과의 연결이 해제되었습니다. 다시 사용하려면 그 PC의 QR 코드를 스캔해 주세요." Galaxy가 브라우저를 여러 개 쓰고 있었을 수 있으므로 끊어진 브라우저 이름(브라우저 목록에 보이는 이름 그대로)을 넣는다. 왜 해제됐는지는 설명하지 않는다. 앱을 열지 않은 사이 여러 개가 끊어졌으면 한 알림에 이름을 이어서 보여준다

> **알고 넘어갈 한계**
>
> 각자 쓰던 휴대폰 두 대로 같은 PC를 번갈아 찍으면 PC가 찍을 때마다 옮겨 간다. 두 휴대폰을 한 PC에 같이 붙이려면 하나가 새로 연결하는 휴대폰이어야 한다. 같은 계정은 어차피 동시에 쓸 수 없으므로(4-2) 실제 불편은 작다. 확장의 "연결 초기화"는 이제 공용 PC 정리용으로만 남는다.

7. 남는 과제 (계속)

과제 ② 재설치하면 폰의 익명 계정이 사라질 수 있다 흔함 · 자동 복구

	iPhone	Android
재설치 후 익명 계정	Firebase 로그인 정보가 Keychain에 남아 **같은 계정으로 돌아오는 경우가 많음**. 보장되지는 않음	항상 사라짐 → 새 익명 계정
복구 방법	확장 QR을 다시 찍으면 새 폰 계정은 "비어 있음" → **확장 계정으로 합류 = 예전 계정 복귀**(그림 5). 복원을 먼저 눌렀어도 4-1 규칙으로 같은 결과	

과제 ③ 폰과 확장을 모두 잃음 드묾 · 감수

백업 기능을 만들지 않기로 했으므로 **새 계정으로 시작**합니다. 구독은 같은 스토어면 "Restore Purchases"로 되찾고, 설정·즐겨찾기는 잃으며, 맛보기는 기기 판정으로 다시 주지 않습니다. 둘 다 지운 사용자는 대부분 그만 쓰려는 사용자이므로 받아들일 만합니다.

과제 ④ 공용 PC · 나눠 쓰기 가끔

- 같은 PC의 QR을 찍은 폰은 모두 같은 계정 → 구독 공유. 폰 수 제한(4-2)으로 범위를 줄인다.
- PC 주인이 바뀌거나 남의 폰을 끊고 싶을 때 → 확장 팝업의 "연결된 폰" 목록과 "연결 초기화".

과제 ⑤ 로그인 코드 정리 확정 · 출시 전이라 전환 없음

- 기존 로그인 사용자가 없으므로 옮길 사람이 없다. 앱의 Apple/Google 로그인, 확장의 Google 로그인, Safari 웹 로그인(`zappit.tv/auth`, `extension_token`), Apple 토큰 해지 코드를 걷어낸다.
- More 탭의 "Log out"은 **"이 폰 연결 해제"**로 바꾼다. 누르면 이 폰은 새 익명 계정으로 돌아가고, 다시 쓰려면 PC의 QR을 찍는다. 확장 팝업의 "연결된 폰" 목록에서도 폰을 해제할 수 있다(양쪽 모두, 확정).
- "내 데이터 삭제" 메뉴는 두지 않는다(확정).

7. 남는 과제 (계속)

과제 ⑥ 확장 60일 미사용 확정 · 로그아웃 대신 연결만 쉼

지금 확장은 앱과 60일 동안 연결되지 않으면 Firebase에서 로그아웃합니다(`background.ts` , `activity.ts`). 새 방식에서 로그아웃하면 확장이 들고 있던 uid(열쇠)를 잃고 QR을 다시 찍어야 합니다. 그래서 **로그아웃은 하지 않고, 60일 동안 앱 연결이 없으면 RTDB 연결만 끊습니다.** 다시 쓰려면 확장 아이콘을 한 번 누르면 됩니다(확정).

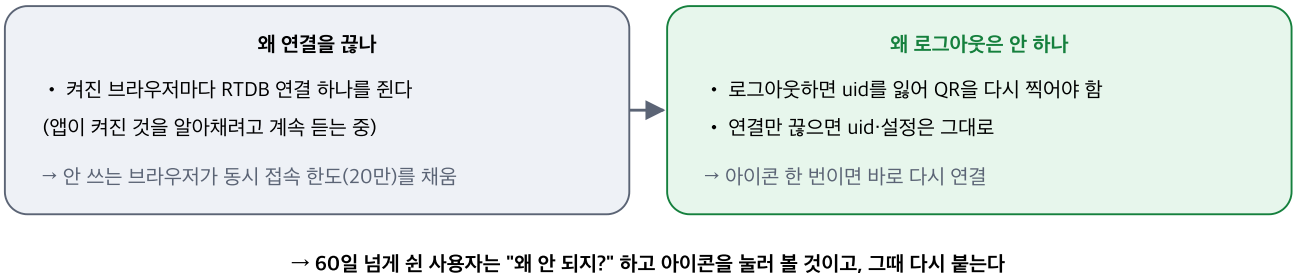


그림 13. 쉬는 브라우저는 연결만 내려놓고, 열쇠(uid)는 계속 들고 있는다.

상황	확장 동작
60일 동안 앱 연결 없음	RTDB 연결을 끊는다(`goOffline`). 로그인은 유지. "쉬는 중" 표시를 `chrome.storage.local` 에 남겨, 브라우저를 다시 켜거나 서비스 워커가 다시 떠도 연결하지 않는다
사용자가 확장 아이콘을 누름	팝업이 열리면 바로 다시 연결(`goOnline`)하고 60일을 새로 센다. 팝업은 평소처럼 연결 상태를 보여 준다
쉬는 동안 앱을 엶	확장이 듣고 있지 않아 브라우저가 나타나지 않는다. 앱의 "브라우저 없음" 화면에 **"PC에서 Zappit 아이콘을 눌러 주세요"** 안내를 넣는다(확정, 실제 문구는 영어로 arb에). 아이콘을 누르면 바로 다시 연결된다

연결별로 언제 끊기나

확장이 쥔 연결은 셋이고, **어느 것도 로그아웃이 아닙니다.** 로그인(uid)은 항상 유지되므로 QR을 다시 찍을 일은 없습니다.

연결	언제 끊나	끊긴 뒤 다시 붙는 방법	로그인(uid)
Ably	앱이 떠나고 10분 뒤 (지금도 이렇게 동작)	앱을 켜면 확장이 RTDB로 알아채고 저절로 다시 연결	유지
RTDB	60일 동안 앱 연결이 없을 때 (새로 추가)	사용자가 확장 아이콘을 눌러야 다시 연결. 앱의 "브라우저 없음" 화면이 안내	유지
Firebase 로그인	끊지 않음 (60일 로그아웃 없앰)	해당 없음	계속 유지

평소에는 확장이 RTDB 하나만 붙잡고 앱이 켜지기를 기다리다가, 앱이 켜지면 Ably를 붙인다. 60일 동안 쉬면 그 RTDB까지 내려놓으므로 앱이 켜져도 확장이 알 수 없고, 아이콘으로 한 번 깨워야 한다.

60일 판정은 지금의 `activity.ts` (12시간마다 확인)를 그대로 쓰고, `signOut` 대신 연결만 끊도록 바꾼다.

7. 남는 과제 (계속)

과제 ⑥ (계속) RTDB 동시 접속 경고 확정

RTDB 동시 접속이 **15만**(한도 20만의 75%)을 넘으면 알림이 오도록 Google Cloud Monitoring에 경고를 건다(지표 `firebasedatabase.googleapis.com/network/active_connections`). 알림이 오면 그때 DB를 여러 개로 나누는 방안을 검토한다. 출시 전에 설정한다.

과제 ⑦ 버려진 익명 계정 1차에는 안 함

앱만 깔고 연결하지 않은 폰, 연결 뒤 남은 확장의 임시 uid(E), PC가 모두 옮겨 가 비게 된 계정이 쌓입니다. 익명 계정은 저장만으로는 무료이고 남는 Firestore 문서도 매우 작아 비용 문제가 없으므로, 1차에는 정리 기능을 만들지 않습니다. 필요해지면 "오래 안 쓴 계정의 탭 목록·재생 기록만 지우기"를 추가합니다. Firebase의 익명 계정 자동 정리(30일)는 쓰는 사용자의 계정까지 지우므로 켜지 않습니다.

8. 결론과 진행 순서

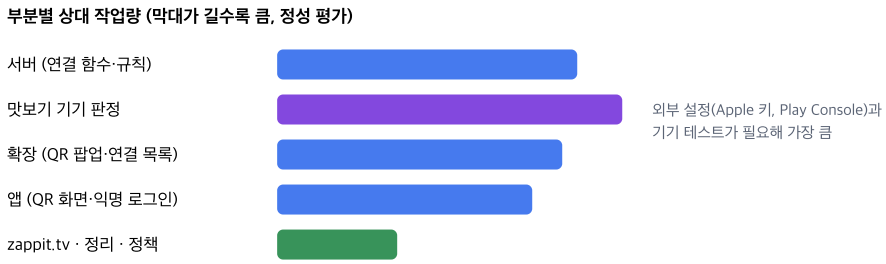
로그인 화면은 없애고, 계정은 "보이지 않게" 남긴다.

사용자는 "앱 켜고 QR 한 번"만 하고, 구독·설정·여러 기기는 지금 구조 그대로 동작합니다. 서버는 Firebase만으로 충분하고, 맛보기는 기기 판정으로 지금보다 단단하게 막습니다.

권장 진행 순서

1. **서버**: `pair_start`, `pair_claim`, `pairings/{code}` (5분 만료, 주인만 읽기), 폰 수 제한. App Check 적용.
2. **맛보기**: `pro_preview_start`에 DeviceCheck·device recall 판정과 폰 표식 추가. DeviceCheck 키, Play Integrity 설정. iOS 네이티브 코드.
3. **확장**: 로그인 코드 제거, 익명 로그인, 팝업 QR, 로그인 표 받으면 전환. "연결된 폰" 목록·"연결 초기화". 60일 미사용 시 RTDB 연결만 끊음.
4. **앱**: 로그인 화면·코드 제거 → QR 스캔 화면. 익명 로그인, 합류 후 `syncPurchases`, 폰 목록 실시간 감시, "브라우저 없음" 화면 아이콘 안내. Log out → "이 폰 연결 해제". DeviceCheck·Play Integrity 네이티브 코드.
5. **zappit.tv**: 앱 링크 설정 파일 두 개, `/p/*` → OneLink 리다이렉트.
6. **정리·정책**: PC가 옮겨 갈 때 떠나는 쪽 목록 정리와 한 번 알림. 스토어 개인정보 항목에 기기 ID 신고. Safari 웹 로그인 페이지(`zappit.tv/auth`) 계획 삭제. RevenueCat 복원 동작 확인 완료. RTDB 동시 접속 15만 경고.

작업량 감(感)



요약 한 장

	지금 (로그인)	새 방식 (보이지 않는 계정)
첫 연결	폰·PC 각각 로그인	QR 한 번
iPhone ↔ Android 구독	가능	가능 (확장이 살아 있으면)
Chrome + Safari	한 계정	한 계정
한쪽 재설치	다시 로그인	QR로 복귀
둘 다 잃음	다시 로그인	새 계정 + 구독 복원 (설정 잃음)
구독 나눠 쓰기	비밀번호 공유 필요	QR로 쉬움 → 폰 수 제한
맛보기 방어	새 Apple/Google 계정이면 뚫림	기기 판정 (초기화해도 막음)
필요한 서버	Firebase	Firebase (+ zappit.tv 링크)